Lab Assignment-3

Name:	**Aalok Kumar Singh**
Roll_no:	**2401201185**
Course_Name/Code:	**Operating System Lab ENCA252**
Program_name:	**BCA (AI & DS)**
Assistant Professor:	**Dr. Sneh Lata Yadav**

Comprehensive Implementation of Page Replacement Algorithms

Introduction

In modern operating systems, virtual memory allows efficient utilization of physical memory. When the number of pages exceeds the available frames, page replacement algorithms are used to decide which page should be replaced.

In this assignment, we implemented multiple page replacement algorithms including FIFO, LRU, Optimal, MRU, and Second Chance. These algorithms help in managing memory efficiently by minimizing page faults.

Objectives

- To understand virtual memory and paging concepts
- To implement various page replacement algorithms
- To calculate the number of page faults
- To compare the efficiency of different algorithms
- To analyze system performance

Tools which are used

- Python 3.x
- VS Code
- Windows OS

Task 1: Input and Page Reference String

Description:-

In this task, we took input from the user including the number of frames and the page reference string. The reference string represents the sequence of page requests made by the system.

Code:-

```
def fifo(pages, frames):
    memory = []
    faults = 0

    for page in pages:
        if page not in memory:
            if len(memory) < frames:
                memory.append(page)
            else:
                memory.pop(0)
                memory.append(page)
            faults += 1
    return faults
```

Screenshot:-

```
C:\Users\AALOK\Desktop\OS_LAB_ASSIGNMENTS\Assignment-3-Bankers-Algorithm>python main.py
Enter number of frames: 3
Enter page reference string: 7 0 1 2 0 3 0 4 2 3 0 3
```

Task 2: FIFO Algorithm

Description:-

FIFO (First-In-First-Out) replaces the oldest page in memory. It maintains a queue of pages and removes the page that entered first.

Code:-

```
def lru(pages, frames):
    memory = []
    faults = 0

    for i in range(len(pages)):
        if pages[i] not in memory:
            if len(memory) < frames:
                memory.append(pages[i])
            else:
                lru_page = min(memory, key=lambda x: pages[:i][::-
1].index(x))
                memory.remove(lru_page)
                memory.append(pages[i])
            faults += 1
    return faults
```

Screenshot:-

```
C:\Users\AALOK\Desktop\OS_LAB_ASSIGNMENTS\Assignment-3-Bankers-Algorithm>python main.py
Enter number of frames: 3
Enter page reference string: 7 0 1 2 0 3 0 4 2 3 0 3

Page Faults:
FIFO: 10
```

Task 3: LRU Algorithm

Description:-

LRU (Least Recently Used) replaces the page that has not been used for the longest period of time. It tracks usage history to determine which page to remove.

Code:-

```
def optimal(pages, frames):
    memory = []
    faults = 0

    for i in range(len(pages)):
        if pages[i] not in memory:
            if len(memory) < frames:
                memory.append(pages[i])
            else:
                future = pages[i+1:]
                replace = -1
                farthest = -1

                for m in memory:
                    if m not in future:
                        replace = m
                        break
                    else:
                        idx = future.index(m)
                        if idx > farthest:
                            farthest = idx
                            replace = m

                memory.remove(replace)
                memory.append(pages[i])
            faults += 1
    return faults
```

ScreenShot

```
C:\Users\AALOK\Desktop\OS_LAB_ASSIGNMENTS\Assignment-3-Bankers-Algorithm>python main.py
Enter number of frames: 3
Enter page reference string: 7 0 1 2 0 3 0 4 2 3 0 3

Page Faults:
LRU: 10
```

Task 4: Optimal Algorithm

Description:-

Optimal page replacement replaces the page that will not be used for the longest time in the future. It provides the minimum number of page faults and serves as a benchmark.

Code:-

```
def optimal(pages, frames):
    memory = []
    faults = 0

    for i in range(len(pages)):
        if pages[i] not in memory:
            if len(memory) < frames:
                memory.append(pages[i])
            else:
                future = pages[i+1:]
                replace = -1
                farthest = -1

                for m in memory:
                    if m not in future:
                        replace = m
                        break
                    else:
                        idx = future.index(m)
                        if idx > farthest:
                            farthest = idx
                            replace = m

                memory.remove(replace)
                memory.append(pages[i])
            faults += 1
    return faults
```

ScreenShot:-

```
C:\Users\AALOK\Desktop\OS_LAB_ASSIGNMENTS\Assignment-3-Bankers-Algorithm>python main.py
Enter number of frames: 3
Enter page reference string: 7 0 1 2 0 3 0 4 2 3 0 3

Page Faults:
Optimal: 7
```

Task 5: MRU Algorithm

Description:-

MRU (Most Recently Used) replaces the most recently used page. It is opposite to LRU and is useful in specific scenarios.

Code:-

```
def mru(pages, frames):
    memory = []
    faults = 0
    recent = []

    for page in pages:
        if page not in memory:
            if len(memory) < frames:
                memory.append(page)
            else:
                memory.remove(recent[-1])
                memory.append(page)
            faults += 1
        if page in recent:
            recent.remove(page)
        recent.append(page)

    return faults
```

ScreenShot:-

```
C:\Users\AALOK\Desktop\OS_LAB_ASSIGNMENTS\Assignment-3-Bankers-Algorithm>python main.py
Enter number of frames: 3
Enter page reference string: 7 0 1 2 0 3 0 4 2 3 0 3

Page Faults:
MRU: 10
```

Task 6: Second Chance Algorithm

Description:-

Second Chance (Clock Algorithm) improves FIFO by giving pages a second chance before replacement. It uses a reference bit to track usage and avoid unnecessary replacements.

Code:-

```python
def second_chance(pages, frames):
    memory = []
    ref = []
    pointer = 0
    faults = 0

    for page in pages:
        if page not in memory:
            while len(memory) == frames and ref[pointer] == 1:
                ref[pointer] = 0
                pointer = (pointer + 1) % frames

            if len(memory) < frames:
                memory.append(page)
                ref.append(1)
            else:
                memory[pointer] = page
                ref[pointer] = 1
                pointer = (pointer + 1) % frames

            faults += 1
        else:
            ref[memory.index(page)] = 1

    return faults
```

ScreenShot:-

```
C:\Users\AALOK\Desktop\OS_LAB_ASSIGNMENTS\Assignment-3-Bankers-Algorithm>python main.py
Enter number of frames: 3
Enter page reference string: 7 0 1 2 0 3 0 4 2 3 0 3

Page Faults:
Second Chance: 9
```

Task 7: Performance Comparison

The performance of all algorithms was compared based on the number of page faults.

From the results:

- Optimal algorithm produced the least page faults
- LRU performed better than FIFO
- MRU showed higher faults in most cases
- Second Chance improved FIFO performance

ScreenShot:-

```
C:\Users\AALOK\Desktop\OS_LAB_ASSIGNMENTS\Assignment-3-Bankers-Algorithm>python main.py
Enter number of frames: 3
Enter page reference string: 7 0 1 2 0 3 0 4 2 3 0 3

Page Faults:
FIFO: 10
LRU: 10
Optimal: 7
MRU: 10
Second Chance: 9
```

Result Analysis

The Optimal algorithm gives the best performance as it has prior knowledge of future references. However, it is not practical in real systems.

LRU is widely used in real systems as it provides a good balance between performance and implementation complexity.

FIFO is simple but may lead to more page faults. MRU is less commonly used.

Second Chance improves FIFO by considering recent usage of pages.

Conclusion

In this assignment, we successfully implemented multiple page replacement algorithms and analyzed their performance. This helped in understanding how operating systems manage memory and reduce page faults.

The comparison of algorithms provided insights into their efficiency and practical applications.